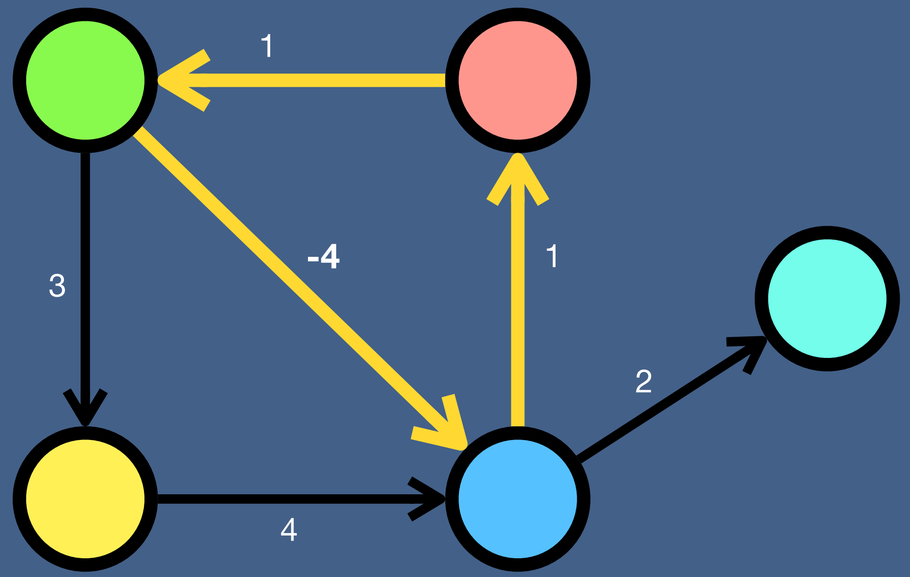
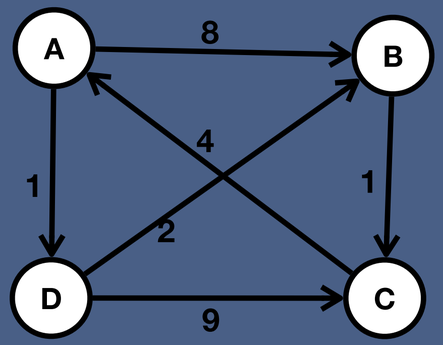
Floyd Warshall Algorithm



Floyd Warshall Algorithm



Given				
	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

Iteration 1				
via A	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	4+8=12	0	4+1=5
D	∞	2	9	0

```
If D[u][v] > D[u][viaX] + D[viaX][v]:
   D[u][v] = D[u][viaX] + D[viaX][v]
```

C -> B

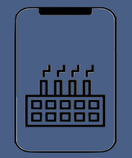
```
If D[C][B] > D[C][A] + D[A][B]:
   D[C][B] = D[C][A] + D[A][B]
```

```
∞ > 4 + 8 = 12
```

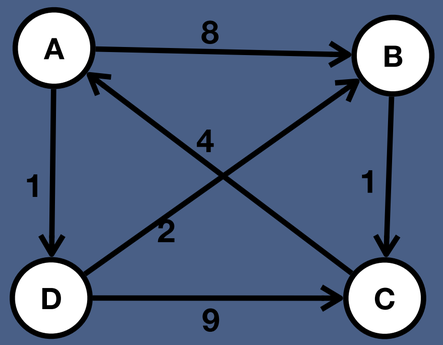
C -> D

```
If D[C][D] > D[C][A] + D[A][D]:
   D[C][D] = D[C][A] + D[A][D]
```

```
∞ > 4 + 1 = 5
```



Floyd Warshall Algorithm




```
If D[u][v] > D[u][viaX] + D[viaX][v]:
   D[u][v] = D[u][viaX] + D[viaX][v]
```

A -> C

```
If D[A][C] > D[A][B] + D[B][C]:
   D[A][C] = D[A][B] + D[B][C]

 ∞ > 8 + 1 = 9
```

D -> C

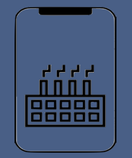
```
If D[D][C] > D[D][B] + D[B][C]:
   D[D][C] = D[D][B] + D[B][C]

 9 > 2 + 1 = 3
```

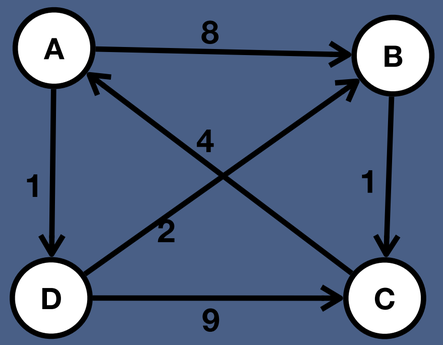
Given				
	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

Iteration 1				
via A	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	4+8=12	0	4+1=5
D	∞	2	9	0

Iteration 2				
via B	A	B	C	D
A	0	8	8+1=9	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	2+1=3	0



Floyd Warshall Algorithm




```
If D[u][v] > D[u][viaX] + D[viaX][v]:
    D[u][v] = D[u][viaX] + D[viaX][v]
```

Given				
	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

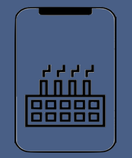
Iteration 1				
via A	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	4+8=12	0	4+1=5
D	∞	2	9	0

Iteration 2				
via B	A	B	C	D
A	0	8	8+1=9	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	2+1=3	0

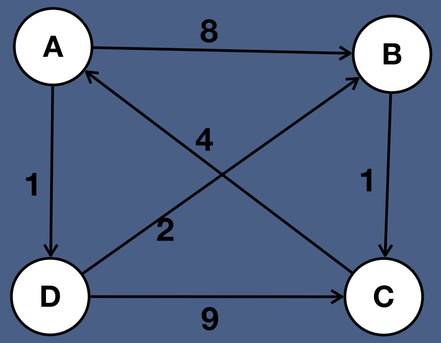
Iteration 3				
via C	A	B	C	D
A	0	8	9	1
B	1+4=5	0	1	1+4+1=6
C	4	12	0	5
D	2+1+4=7	2	3	0

Iteration 4				
via D	A	B	C	D
A	0	1+2=3	9	1
B	5	0	1	6
C	4	4+1+2=7	0	5
D	7	2	3	0

Final answer				
	A	B	C	D
A	0	3	4	1
B	5	0	1	6
C	4	7	0	5
D	7	2	3	0



Why Floyd Warshall Algorithm?



E

Given				
	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

Iteration 1				
via A	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	4+8=12	0	4+1=5
D	∞	2	9	0

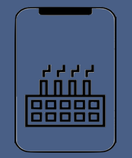
Iteration 2				
via B	A	B	C	D
A	0	8	8+1=9	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	2+1=3	0

Iteration 3				
via C	A	B	C	D
A	0	8	9	1
B	1+4=5	0	1	1+4+1=6
C	4	12	0	5
D	3+4=7	2	3	0

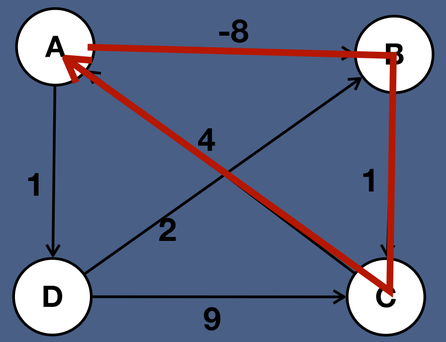
Iteration 4				
via D	A	B	C	D
A	0	1+2=3	3+1=4	1
B	5	0	1	6
C	4	5+2=7	0	5
D	7	2	3	0

Final answer				
	A	B	C	D
A	0	3	4	1
B	5	0	1	6
C	4	7	0	5
D	7	2	3	0

- **The vertex is not reachable**
- **Two vertices are directly connected**
  - This is the best solution
  - It can be improved via other vertex
- **Two vertices are connected via other vertex**

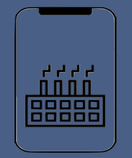


Floyd Warshall negative cycle



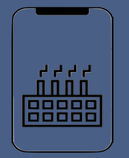
-8+1+4=-3

- To go through cycle we need to go via negative cycle participating vertex at least twice
- FW never runs loop twice via same vertex
- Hence, FW can never detect a negative cycle



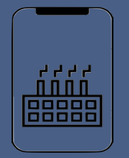
Which algorithm to use for APSP?

Graph Type	BFS	Dijkstra	Bellman Ford	Floyd Warshall
Unweighted – undirected	OK	OK	OK	OK
Unweighted – directed	OK	OK	OK	OK
Positive – weighted – undirected	X	OK	OK	OK
Positive – weighted – directed	X	OK	OK	OK
Negative – weighted – undirected	X	OK	OK	OK
Negative – weighted – directed	X	OK	OK	OK
Negative Cycle	X	X	OK	X



Which algorithm to use for APSP?

	BFS	Dijkstra	Bellman Ford	Floyd Warshall
Time complexity	$O(V^3)$	$O(V^3)$	$O(EV^2)$	$O(V^3)$
Space complexity	O(EV)	O(EV)	$O(V^2)$	$O(V^2)$
Implementation	Easy	Moderate	Moderate	Moderate
Limitation	Not work for weighted graph	Not work for negative cycle	N/A	Not work for negative cycle
Unweighted graph	OK	OK	OK	OK
	Use this as time complexity is good and easy to implement	Not use as priority queue slows it	Not use as time complexity is bad	Can be used
Weighted graph	X	OK	OK	OK
	Not supported	Can be used	Not use as time complexity is bad	Can be preferred as implementation easy
Negative Cycle	X	X	OK	X
	Not supported	Not supported	Use this as others not support	No supported



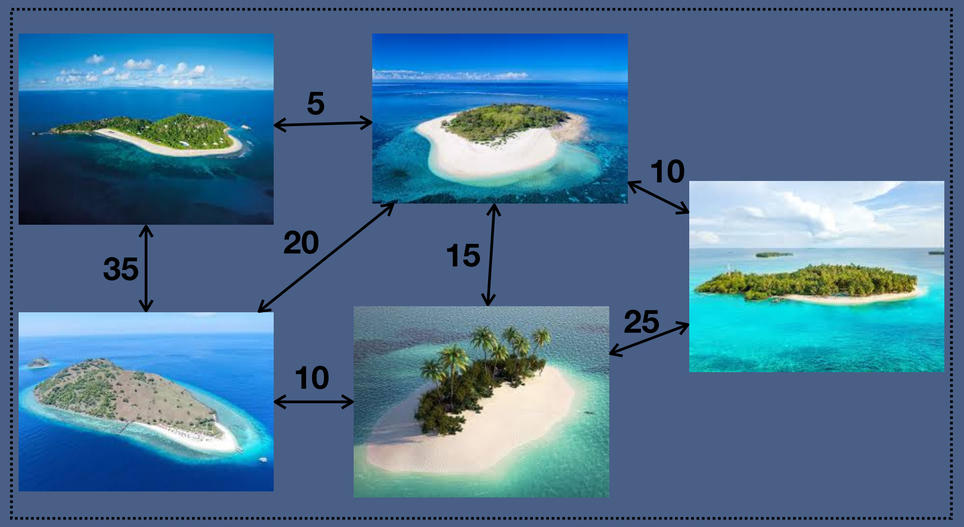
Minimum Spanning Tree

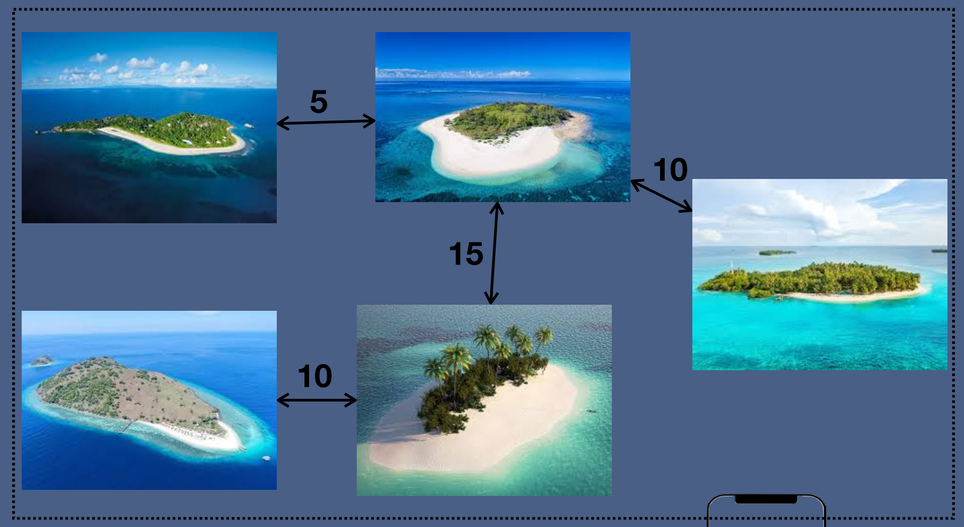
A Minimum Spanning Tree (MST) is a subset of the edges of connected, weighted and undirected graph which :

- Connects all vertices together
- No cycle
- Minimum total edge

Real life problem

- Connect five island with bridges
- The cost of bridges between island varies based on different factors
- Which bridge should be constructed so that all islands are accessible and the cost is minimum

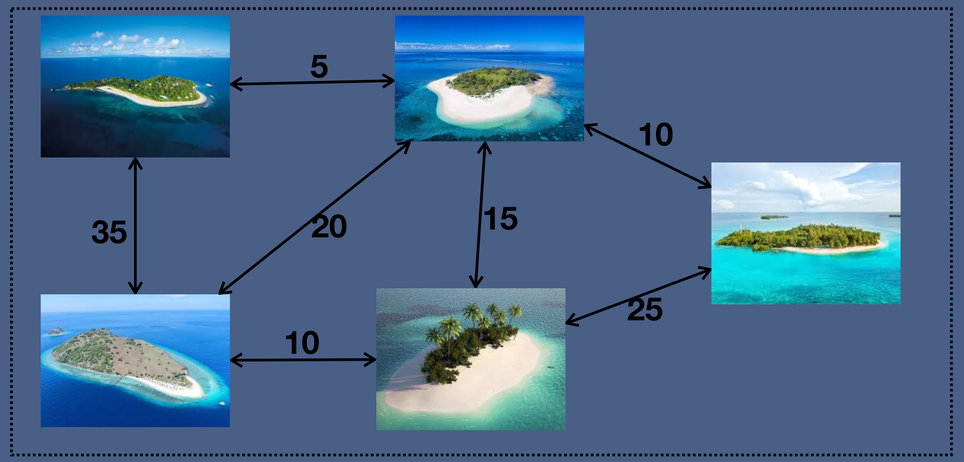




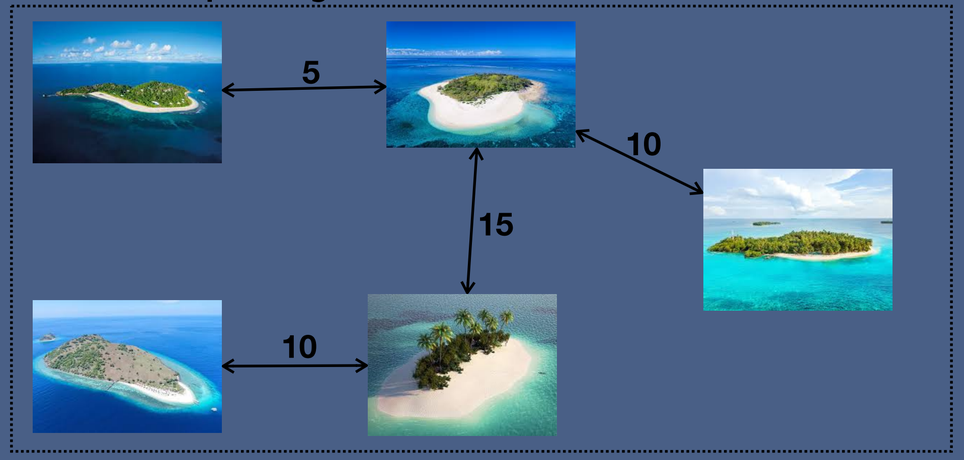
AppMillers
www.appmillers.com

Minimum Spanning Tree

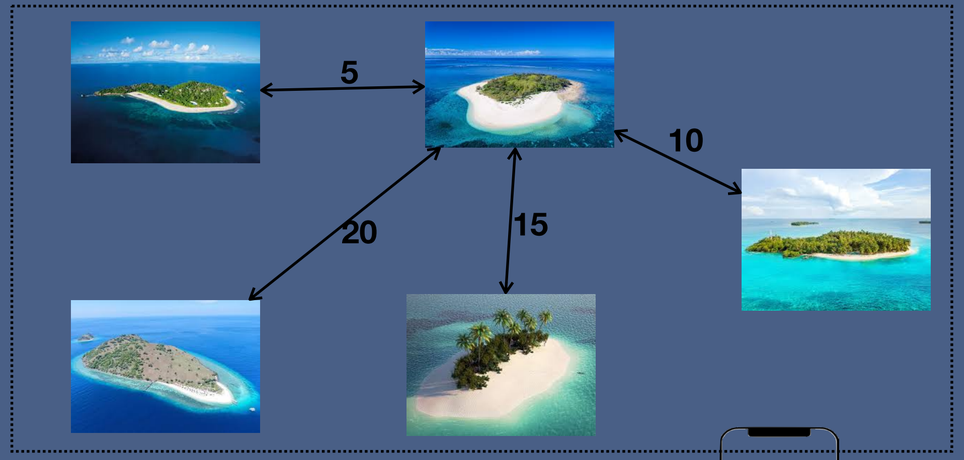
Real life problem



Minimum spanning Tree



Single source shortest path

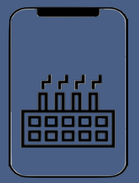


AppMillers
www.appmillers.com

Disjoint Set

It is a data structure that keeps track of set of elements which are partitioned into a number of disjoint and non overlapping sets and each sets have representative which helps in identifying that sets.

- Make Set
- Union
- Find Set



Disjoint Set

makeSet(N) : used to create initial set

union(x,y): merge two given sets

findSet(x): returns the set name in which this element is there